# BLOCKCHAIN Experiment No: 9


## Manas Patil
## D20A/24


**AIM:** Create an ERC 20 Token in Remix IDE

**Task to be performed :**

1. Go to **LearnETH** Tutorials provided by **Remix IDE**
2. Explore through **Solidity ERC 20 Token Course**
3. Complete all the **6 chapters** provided with the Course
4. As a part of this experiment, we need to create a Metamask Wallet as described in the document
5. Fund Your Wallet : Sepolia Testnet (0.5 ETH per day) / RSK Testnet (0.05 RBTC per day) / try the Google Cloud Web3 faucet to fund your wallet by connecting it the Testnet
6. Deploy the ERC 20 Token to the selected testnet

**Theory :**

ERC-20 (Ethereum Request for Comments 20) is a token standard used to create and manage fungible tokens on the Ethereum blockchain. Fungible tokens are interchangeable, meaning every unit of a particular token has the same value and properties.

Solidity is a programming language used to develop smart contracts, while Remix IDE is an online platform used to write, compile, deploy, and test these contracts.

An ERC-20 smart contract provides six important functions:

1. totalSupply(): Returns the total number of tokens issued.
2. balanceOf(): Returns the token balance of an account.
3. transfer(): Transfers tokens from one account to another.
4. approve(): Authorizes another account to spend tokens.
5. allowance(): Returns the remaining approved spending amount.
6. transferFrom(): Transfers tokens on behalf of another account using an approved allowance.

It also includes two important events: `Transfer` and `Approval`, which record token transfers and spending approvals.

In this experiment, a custom ERC-20 token named Manas Token (MNS) was created using Solidity in Remix IDE. The token was configured with an initial supply of 1,000,000 tokens and 18 decimal places. MetaMask was used to connect Remix IDE to the Sepolia Testnet, where the smart contract was deployed

using test ETH.

After deployment, the token details, total supply, wallet balance, and transfer functionality were verified. The experiment also covered the differences between ERC-20 and ERC-721 standards and the Mintable, Burnable, and Pausable extensions.

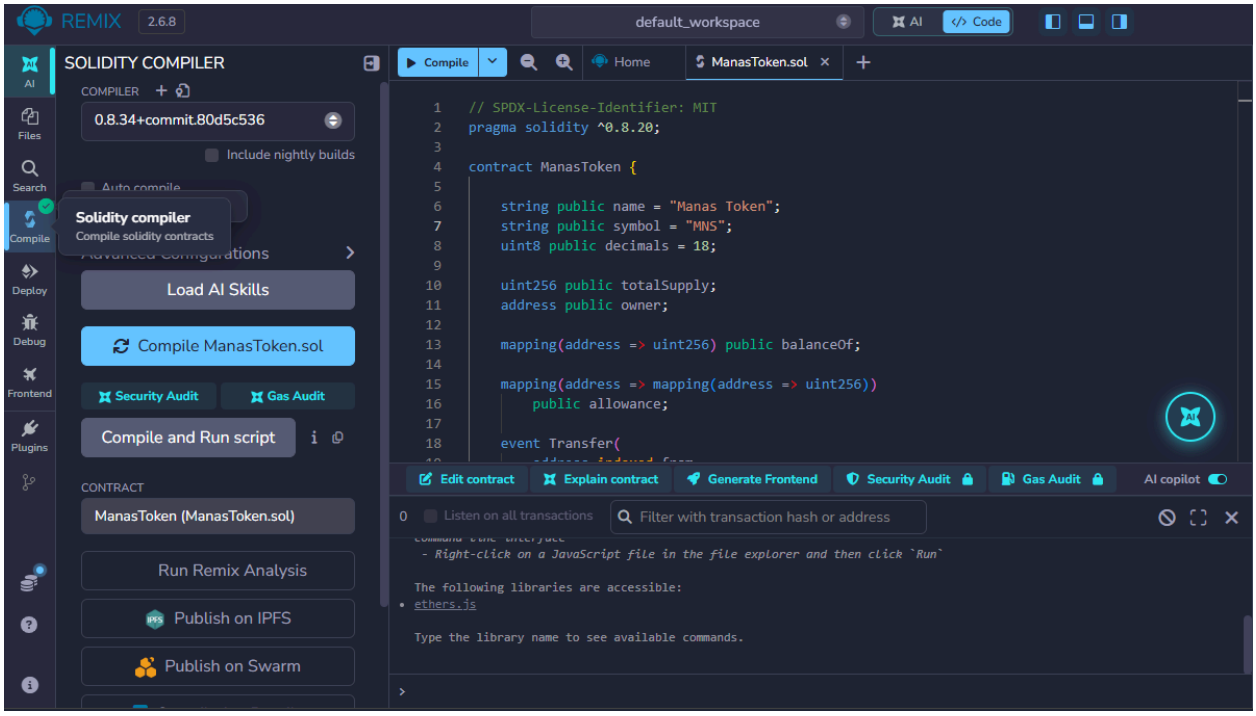
## Procedure :

### Step 1: Open the website

Open the official Remix IDE:

Remix IDE

### Step 2: Create a Solidity file




### Step 3 : Write the ERC-20 Smart Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract ManasToken {

    string public name = "Manas Token";
    string public symbol = "MNS";
    uint8 public decimals = 18;

    uint256 public totalSupply;
```

```
    address public owner;

    mapping(address => uint256) public balanceOf;

    mapping(address => mapping(address => uint256))
    public allowance;

    event Transfer(
        address indexed from,
        address indexed to,
        uint256 value
    );

    event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );

    constructor() {
        owner = msg.sender;

        uint256 initialSupply = 1000000 * (10 ** uint256(decimals));

        totalSupply = initialSupply;
        balanceOf[msg.sender] = initialSupply;

        emit Transfer(address(0), msg.sender, initialSupply);
    }

    function transfer(
        address to,
        uint256 value
    ) public returns (bool) {

        require(to != address(0), "Invalid recipient");

        require(balanceOf[msg.sender] >= value, "Insufficient balance");

        balanceOf[msg.sender] -= value;
        balanceOf[to] += value;

        emit Transfer(msg.sender, to, value);

        return true;
    }

    function approve(
        address spender,
        uint256 value
    ) public returns (bool) {

        require(spender != address(0), "Invalid spender");

        allowance[msg.sender][spender] = value;

        emit Approval(msg.sender, spender, value);

        return true;
    }

    function transferFrom(
        address from,
        address to,
        uint256 value
    ) public returns (bool) {

        require(to != address(0), "Invalid recipient");
        require(
            balanceOf[from] >= value,
            "Insufficient balance"
        );
        require(
            allowance[from][msg.sender] >= value,
            "Allowance exceeded"
        );
```

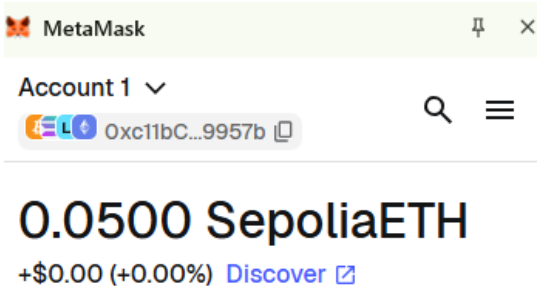
```
        emit Transfer(from, to, value);
    allowance[from][msg.sender] -= value;
    balanceOf[from] -= value;                return true;
    balanceOf[to] += value;                }
```

**Step 4: Compile the Smart Contract**

Select the Solidity compiler

1. Click the Solidity Compiler icon on the left side of Remix.
2. Select compiler version `0.8.20` or another compatible version from the `0.8.x` series.
3. Ensure that the version satisfies the program's `pragma` statement.
4. Keep the default optimization settings for this experiment.
5. Click Compile ManasToken.sol.

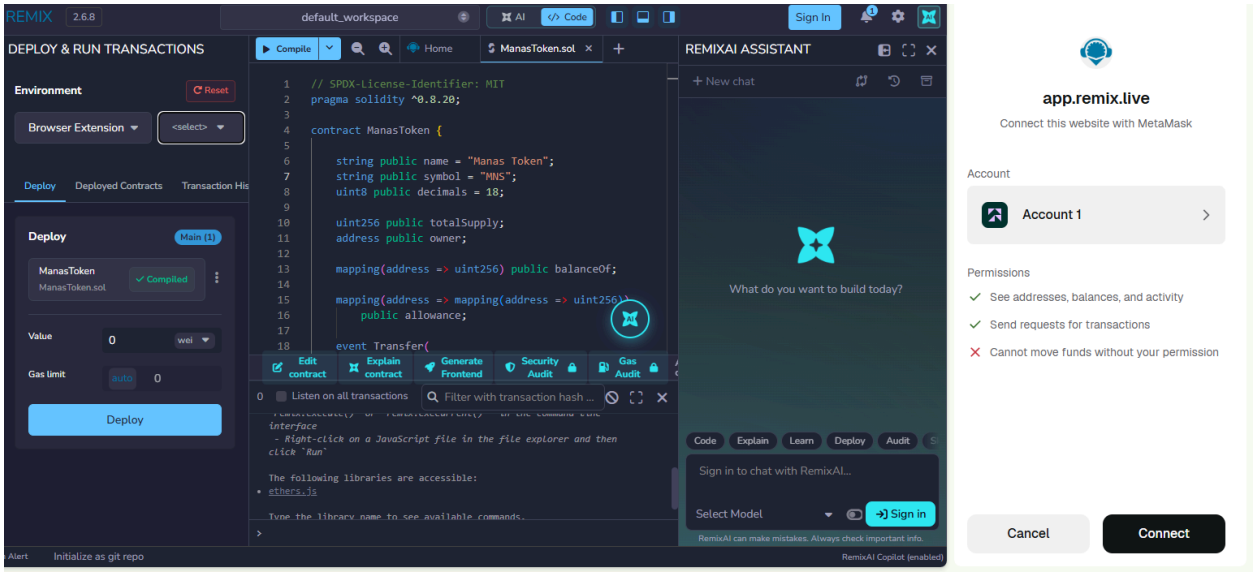
**Step 5: Create Your MetaMask Wallet and Fund Your Wallet with Test ETH**




**Step 6: Deploy the ERC-20 Token to Sepolia**

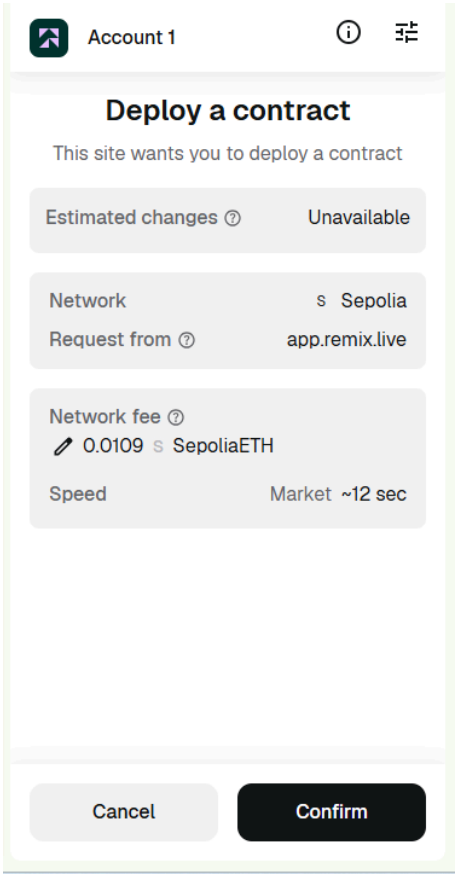
**Connect Remix to MetaMask**

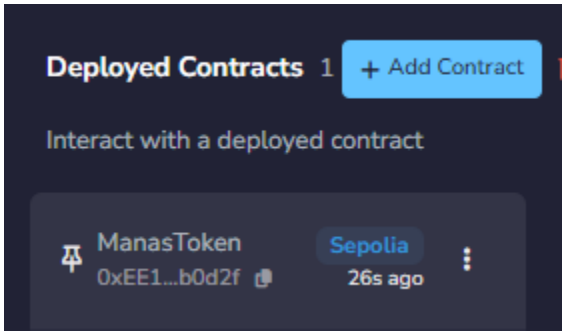
1. Return to the Remix IDE.
2. Click Deploy & Run Transactions on the left.
3. Find the Environment dropdown.
4. Select Browser Extension or the equivalent injected-provider option available in your Remix version.
5. MetaMask should open and ask you to connect.
6. Select your test account and approve the connection.
7. Check that MetaMask is still connected to the Sepolia Testnet.




**Deploy the contract :** In the Remix deployment panel

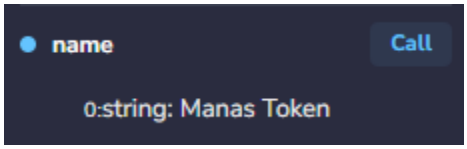
1. Select `ManasToken` in the contract dropdown.
2. Confirm that the deployment environment uses your connected wallet.
3. The contract does not require any constructor arguments, so you do not need to enter anything.
4. Click Deploy.
5. MetaMask will open a transaction confirmation window.
6. Check the network and estimated gas fee.
7. Confirm the transaction.



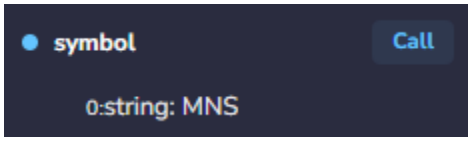



**Step 7: Test the ERC-20 Token**

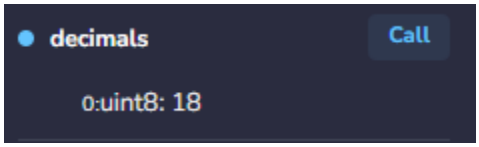
- **Check the token name**



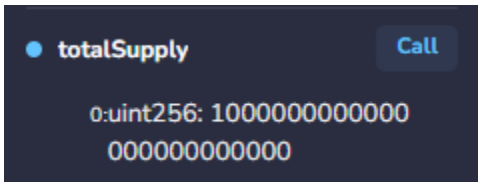

- **Check the token symbol**




- **Check the decimals**




- **Check the total supply**




This represents 1,000,000 MNS tokens.

- Check your wallet balance

1. Copy your MetaMask account address.
2. Paste it into the input field for `balanceOf`.
3. Click the button.

Expected result:

```
1000000000000000000000000
```

This is the initial token balance of the wallet that deployed the contract.

- **Test a token transfer**

For this test, you need a second account address. You can create a second account in MetaMask or use another test wallet you control.

Suppose you want to transfer 100 MNS tokens.

In the `transfer` function, enter:

- `to`: The second account's public address.
- `value`: `100000000000000000000`

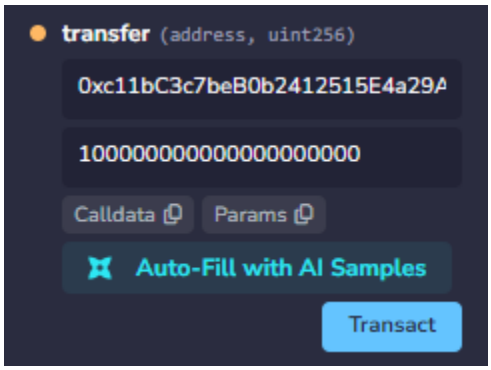
Why such a large number?

Because the contract stores token amounts in their smallest units, and each whole token is represented by $10^{18}$ units.

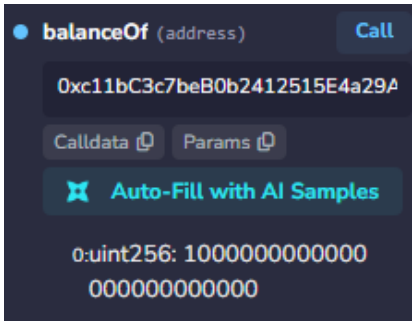
Therefore:

$$100MNS = 100 \times 10^{18} units$$

Click `transfer` and confirm the transaction in MetaMask.




After the transaction succeeds, call `balanceOf` for the recipient.



The recipient now holds 100 MNS tokens.

## Conclusion :

In this experiment, we successfully created and deployed an ERC-20 token smart contract using Solidity in Remix IDE on the Sepolia Testnet. We tested token transfers, approvals, balances, and other essential ERC-20 functions. We also understood the differences between ERC-20 and ERC-721 standards and studied Mintable, Burnable, and Pausable extensions. Thus, the experiment provided practical knowledge of creating, deploying, and interacting with blockchain tokens.